Árboles de Regresión y Clasificación

Métodos no métricos

Antonio M. López Rodríguez

Clasificación

Ejemplo: Se ha de decidir si un pez recién pescado es una dorada o una lubina.



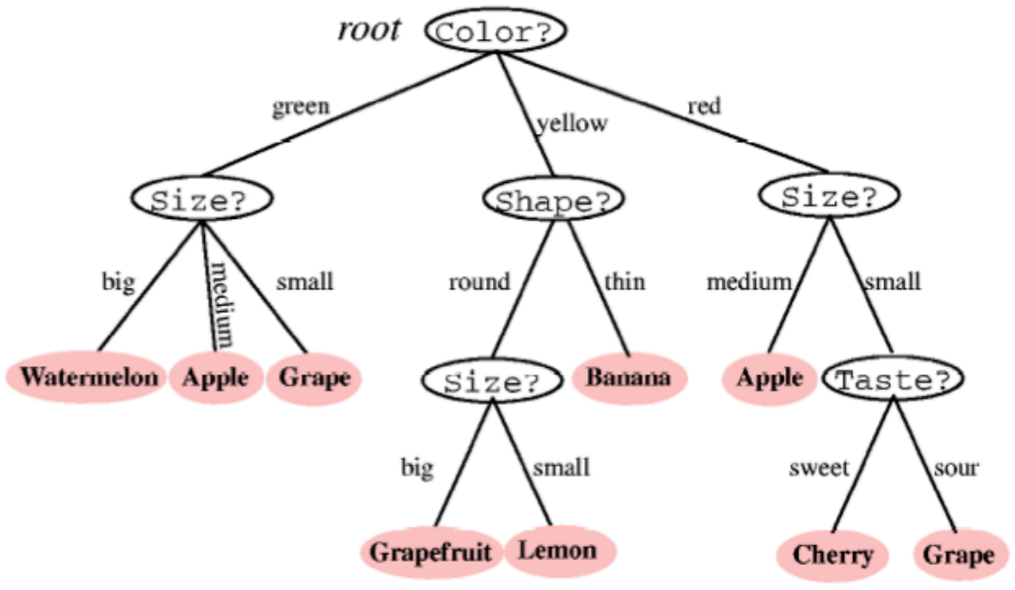
- Hasta ahora hemos considerado patrones representados por vectores de características formados por números reales (o enteros): peso, brillo, …
- Para este tipo de datos resulta “sencillo” definir una medida de similitud entre un elemento y unos patrones determinados, la cual usamos para desarrollar el clasificador.
- ¿Qué pasa cuando:
  - queremos usar características donde no se puede cuantificar la similitud entre las características, o
  - las características no tienen una representación numérica?

Ejemplo: frutas

- Características:
  - Color:
    - Las manzanas rojas no son más parecidas a las verdes que la amarillas.
  - Forma.
  - Sabor.
  - Etc.
- Nuestro vector de características no es un vector de números, sino una lista de valores cualitativos.
  - {verde, redondeada, dulce, …}

Árboles de Decisión

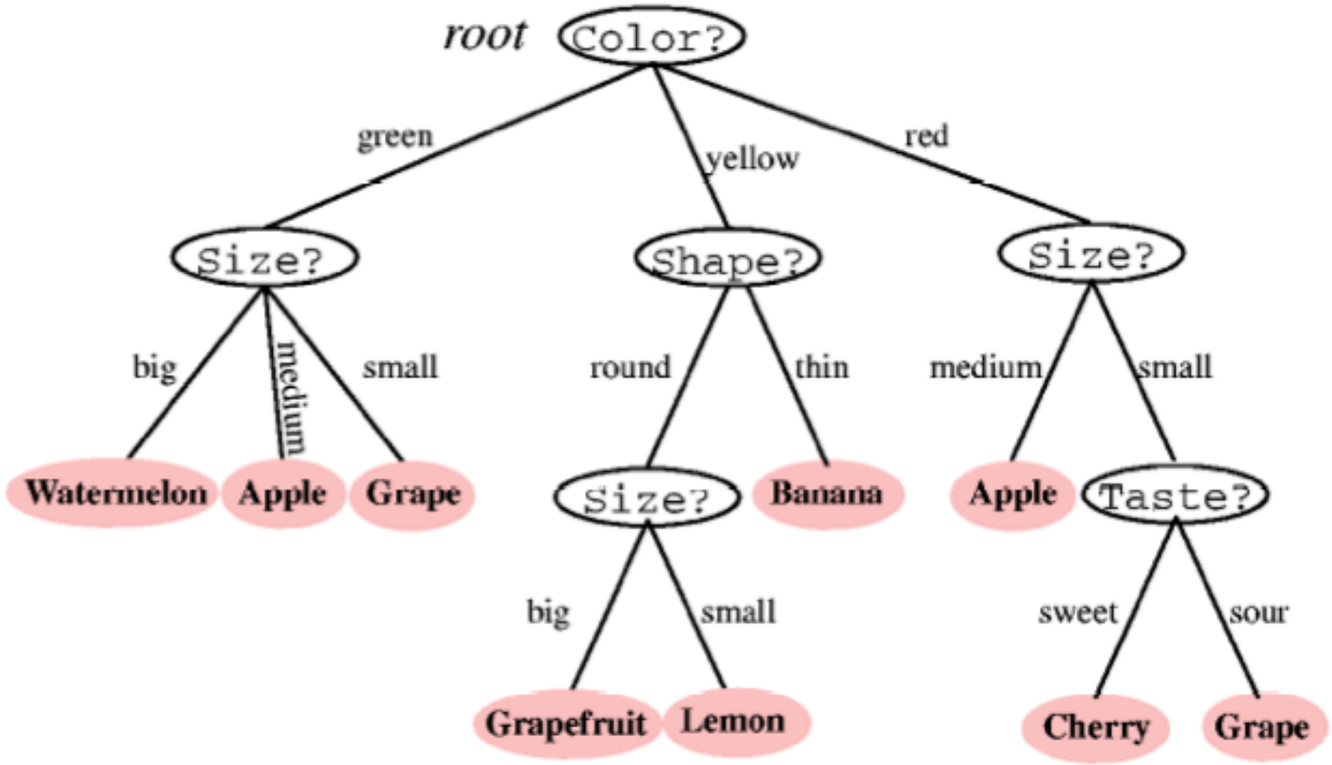
- Conjunto de reglas basadas en valores particulares de las características que permiten realizar la clasificación.
- Estas reglas pueden representarse fácilmente mediante un árbol.



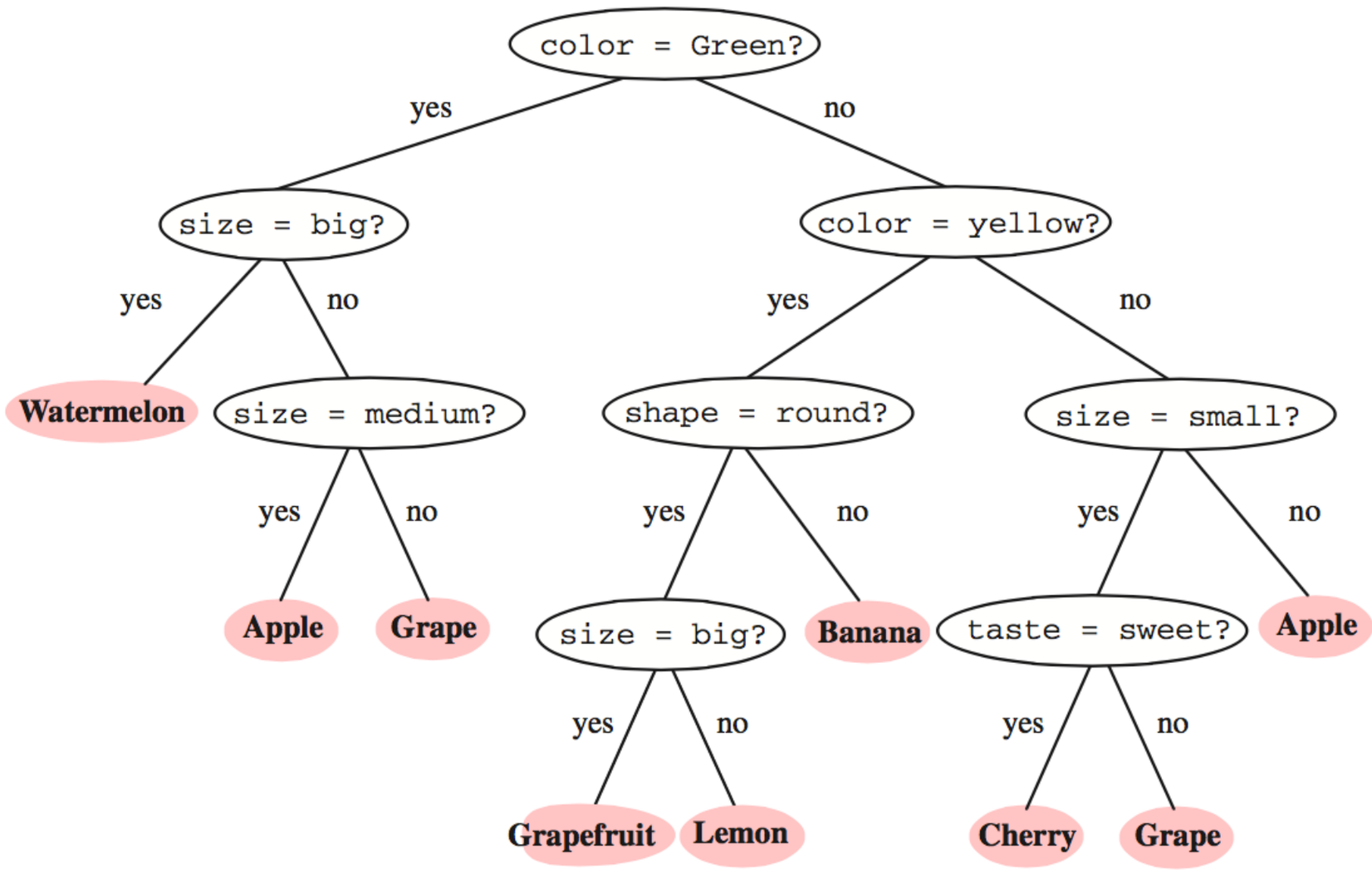
- Las preguntas pueden formularse respecto a una o varias características.
- La respuesta puede ser:
  - Si/No
  - Un valor de un conjunto finito.

Ejemplo

- Si sabemos que una fruta es
  - {grande, amarilla, redondeada},
- ¿de qué fruta se trata?



Equivalencia con árboles binarios



Machine Learning en Árboles de Decisión

- Tenemos un conjunto de ejemplos de las características junto con la clase verdades a que pertenecía el objeto de que se extrajeron.
- ¿Cómo podemos "crear" un árbol de decisión en base a esa información?

Day	Outlook	Temp.	Humidity	Wind	Play Tennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Weak	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Strong	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

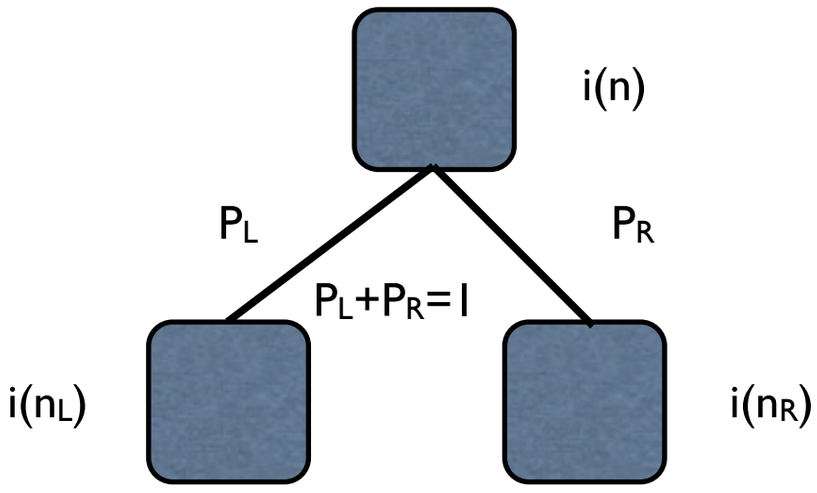
¿Es adecuado jugar al tenis cuando …?

- Vamos a ir planteándonos preguntas referidas a una característica que nos permitan ir dividiendo los ejemplos en subconjuntos cada vez más "puros".
- Un subconjunto sería puro si la decisión para todos los datos es única.
- Estas reglas las usaremos después para clasificar nuevos ejemplos.

Criterio para subdividir (CART)

- Antes de dividir tenemos una determinada impureza de los datos “asignados” a un nodo (n).
- Si realizamos una división de los datos (n_L, n_R) tendremos una impureza en cada subconjunto.
- La mejor división (greedy) será aquella que maximice la disminución de la impureza:

$$\Delta i(n) = i(n) - P_L i(N_L) - p_R i(N_R)$$



- i(n): impureza del nodo n.
- $P_{L/R}$: proporción de muestras en nodo n que pasan a la rama L/R.
- Δi = Ganancia de información

Gain Ratio

- El método de partición anterior puede extenderse a particiones de mayor aridad:

$$\Delta i(n) = i(N) - \sum_k P_K i(N_k)$$

- No obstante esto favorece un número elevado de particiones (en el extremo, si partiendo de un nodo se permite una partición que asigne un único ejemplo a cada clase en el siguiente nivel, la ganancia de información será máxima por ser la impureza de los nodos resultantes nula.
- En este caso se suele dividir según aquel atributo que maximice el ratio de impureza de ganancia:

$$\Delta i_B(n) = \frac{\Delta i(n)}{-\sum_k P_K log_2(P_k)}$$

$$P_k = \frac{\text{numero elementos que caen en la particion } i}{\text{numero de elementos totales}}$$

- El denominador (“Split information”) busca divisiones equilibradas en el número de elementos de cada partición.
- Otras opciones son penalizar el número máximo de particiones o evitar particiones con tamaños muy pequeños.

Gain Ratio

- Supongamos que tenemos un conjunto de datos con 10000 clientes clasificados en dos clases:
  - Compra (6000 clientes).
  - No Compra (4000 clientes).
  - $i(N) = -P(\text{Compra})\log_2(P(\text{Compra})) - P(\text{No Compra})\log_2(P(\text{No Compra}))$
  - $i(N) = = -\frac{6000}{10000}\log_2\left(\frac{6000}{10000}\right) - \frac{4000}{10000}\log_2\left(\frac{4000}{10000}\right) = 0.97$
- Supongamos dos opciones para dividir los datos:
  - Opción 1: por “Edad”.
    - Joven: 4000 clientes (3000 Compra, 1000 No Compra)
    - Adulto: 3000 clientes (2000 Compra, 1000 No Compra).
    - Mayor: 3000 clientes (1000 Compra, 2000 No Compra).
    - $i(\text{Joven}) = -\frac{3000}{4000}\log_2\left(\frac{3000}{4000}\right) - \frac{1000}{4000}\log_2\left(\frac{1000}{4000}\right) = 0.81$
    - i(Adulto) = 0.92
    - i(Mayor) 0.92
    - $\Delta i(\text{Edad}) = i(N) - \left(\frac{4000}{10000}\cdot 0.81 + \frac{3000}{10000}\cdot 0.92 + \frac{3000}{10000}\cdot 0.92\right)$ =0.114
    - Información de División:
      - $\text{Split Information} = -\left(\frac{4000}{10000}\log_2\left(\frac{4000}{10000}\right) + \frac{3000}{10000}\log_2\left(\frac{3000}{10000}\right) + \frac{3000}{10000}\log_2\left(\frac{3000}{10000}\right)\right) = 1.57$
      - $\text{Gain Ratio} = \frac{\Delta i(\text{Edad})}{\text{Split Information}} = \frac{0.114}{1.57} = 0.073$
- Opción 2: Por Persona
  - $\Delta i(\text{Persona}) = i(N) - 0 = 0.97$
  - Información de División (10000 particiones, una por persona):
    - $\text{Split Information} = -\sum_{k=1}^{10000}\frac{1}{10000}\log_2\left(\frac{1}{10000}\right) = 13.29$
    - $\text{Gain Ratio} = \frac{\Delta i(\text{Identificador Único})}{\text{Split Information}} = \frac{0.97}{13.29} = 0.073$

Medidas de impureza

- Suponemos que en el nodo *n* hay *j* clases diferentes con una proporción $P(w_j)$:
  - Missclassification error:

$$i(n) = 1 - \max_j P(w_j)$$

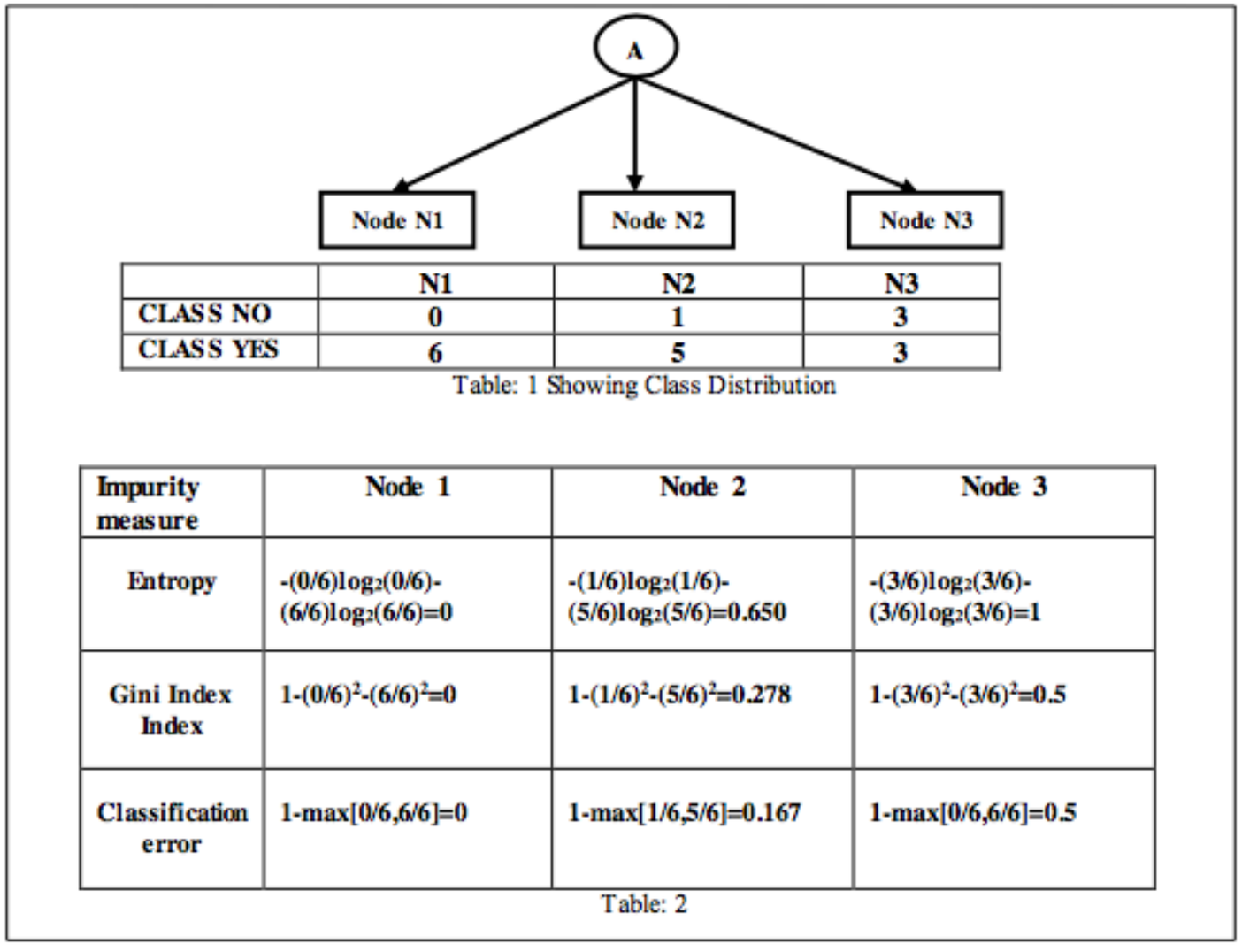
  - Impureza de Gini (CART):

$$i(n) = 1 - \sum_j P(w_j)^2$$

  - Entropía (ID3, C4.5):

$$i(n) = -\sum_j P(w_j) log_2(P(w_j))$$

Medidas de impureza



	N1	N2	N3
CLASS NO	0	1	3
CLASS YES	6	5	3

Table: 1 Showing Class Distribution

Impurity measure	Node 1	Node 2	Node 3
Entropy	$-(0/6)\log_2(0/6)-(6/6)\log_2(6/6)=0$	$-(1/6)\log_2(1/6)-(5/6)\log_2(5/6)=0.650$	$-(3/6)\log_2(3/6)-(3/6)\log_2(3/6)=1$
Gini Index Index	$1-(0/6)^2-(6/6)^2=0$	$1-(1/6)^2-(5/6)^2=0.278$	$1-(3/6)^2-(3/6)^2=0.5$
Classification error	1-max[0/6,6/6]=0	1-max[1/6,5/6]=0.167	1-max[0/6,6/6]=0.5

Table: 2

Figure: 2 Computing Impurity Measures

Medidas de impureza

- Cada una con sus propiedades. Ninguna es mejor en términos absolutos que las otras.

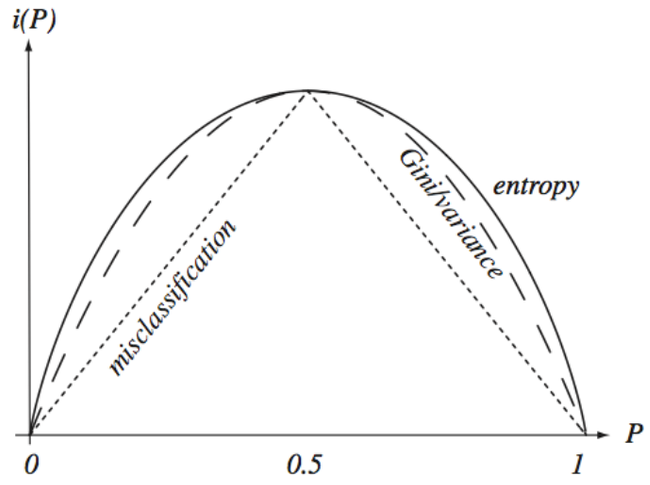


Figure 8.4: For the two-category case, the impurity functions peak at equal class frequencies and the variance and the Gini impurity functions are identical. To facilitate comparisons, the entropy, variance, Gini and misclassification impurities (given by Eqs. 1 – 4, respectively) have been adjusted in scale and offset to facilitate comparison; such scale and offset does not directly affect learning or classification.

Atributos continuos

- Supongamos que un atributo C es continuo:
  - Ordenémoslo.
  - Para cada pareja de valores $[A_j, A_{j+1}]$ pensemos en la condición $A<(A_j+A_{j+1})/2$
  - Calculemos la ganancia de información con cada posible partición.
  - Quedémonos con aquella que la maximiza.

Interrupción de la ramificación

- Si hacemos crecer el árbol hasta reducir la impureza de los nodos a cero, generalmente tendremos un árbol sobreentrenado (en el caso extremo cada nodo se corresponde con un dato, por lo que el árbol no es otra cosa que una tabla de búsqueda).
- Si paramos muy pronto, el árbol no se habrá ajustado ni siquiera a los datos de entrenamiento, por lo que tendremos un caso subentrenado.
- Algunos posibles criterios:
  - Detener el proceso de splitting por validación cruzada (no podremos usar todos los daos disponibles era entrenar).
  - Poner un umbral en la reducción de impureza para continuar.
  - Limitar el número mínimo de casos asociados a un nodo.
  - Crear test de complejidad (algo similar a la regularización):
  - …

$$\alpha \cdot size + \sum_{leaf\ nodes} i(N),$$

Poda

- Es una alternativa a detener la ramificación:
  - Dejamos crecer el árbol a su nivel máximo.
  - Realizamos una poda posterior: se examinan pares de nodos hoja con un mismo antecesor para ver si se pueden fusionar (el incremento en la impureza se define tolerable).
  - Una ventaja es que usamos todos los datos para el entrenamiento.

Asignación de etiquetas a nodos

- Se ha de tener en cuenta siempre que un nodo presente impurezas.
- Lo habitual es asignar la etiqueta correspondiente a la clase con mayor presencia.

ID3

- Atributos categóricos.
- Árboles n-arios.
- El árbol se desarrolla en su máximo nivel y no se definen mecanismos de poda.
- El número de niveles coincidirá con el número de características.

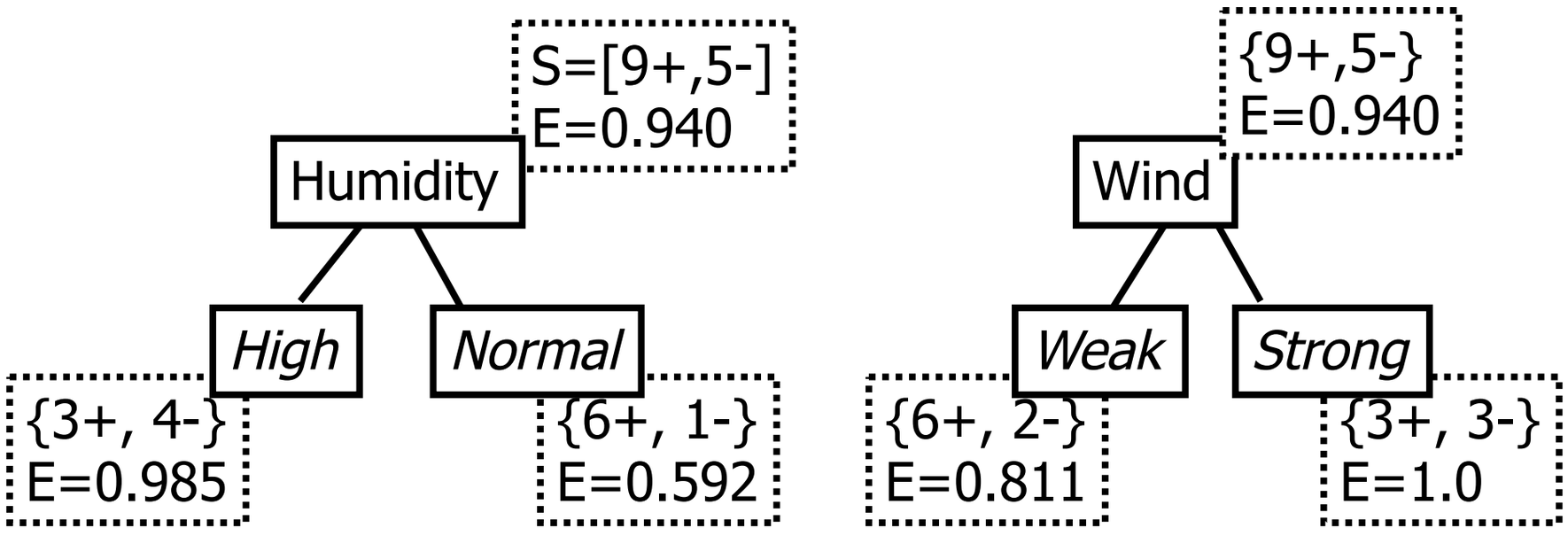
C4.5

- Sucesor de ID3.
- Árboles n-arios.
- Atributos categóricos y numéricos.
- Usa el ratio de ganancia.
- Incorpora heurísticos de poda.
- Una diferencia fundamental con CART son sus mecanismos para manejar datos con atributos desconocidos.

Otros tipos de árbol

- C5.0, CHAID, MARS.

Ejemplo (entropía): tenis



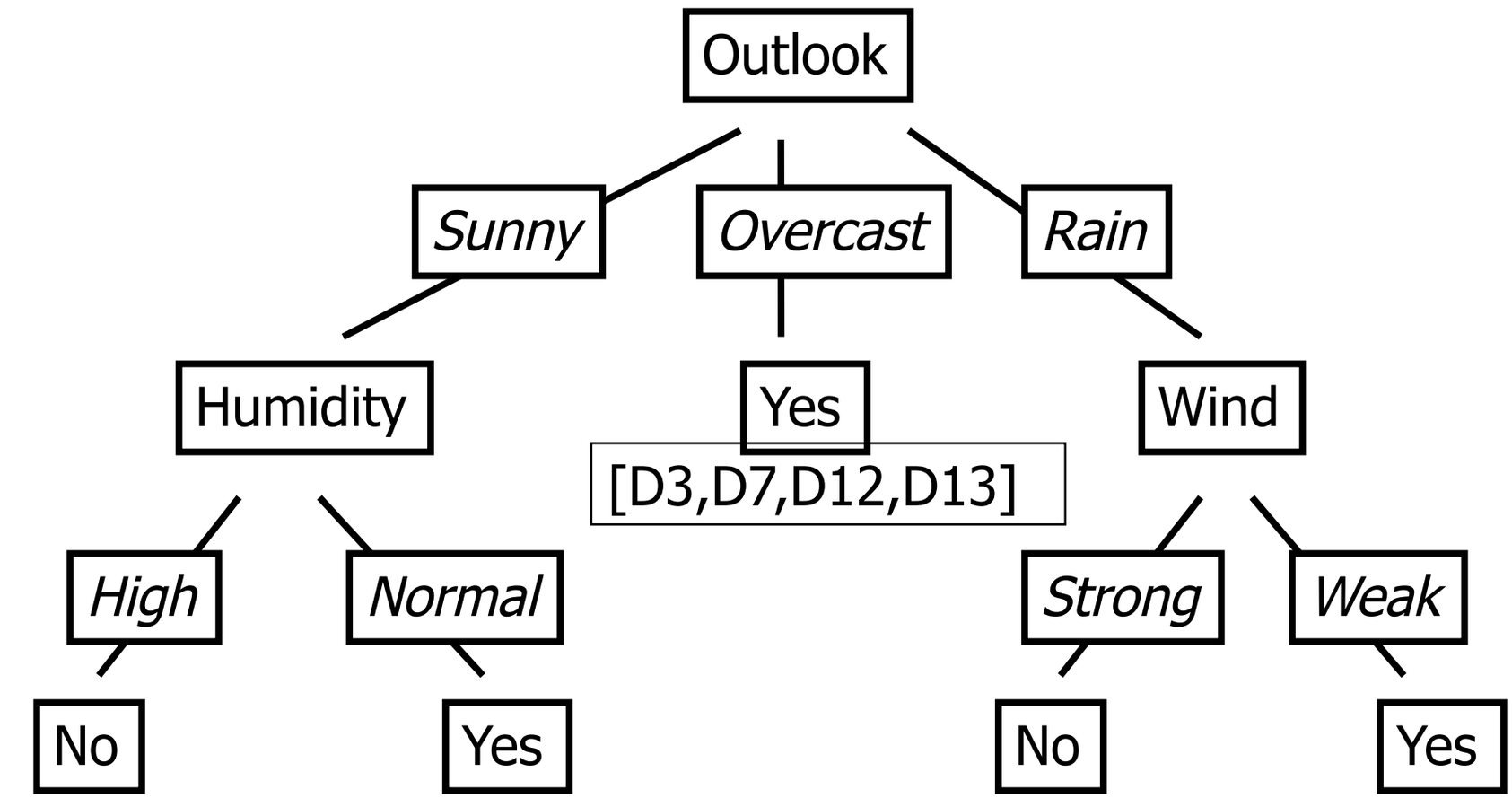
Gain(S,Humidity)
=0.940-(7/14)*0.985
– (7/14)*0.592
=0.151

Gain(S,Wind)
=0.940-(8/14)*0.811
– (6/14)*1.0
=0.048

Gain(S,Outlook) =0.246
Gain(S,Humidity) =0.151
Gain(S,Wind) =0.048
Gain(S,Temperature) =0.029

La impureza en cada nodo se evalúa respecto a los posibles valores objetivo de la clasificación ("Yes","No")

Ejemplo: tenis



Ventajas

- Interpretabilidad.
- Coste computacional.
- Permiten manejar conjuntos elevados de características.
- C4.5 da mecanismos para manejar missing features.

Inconvenientes

- Es un método greedy.
- Las superficies de decisión pueden ser difíciles de inducir.

Otros tipos de árbol

- C5.0, CHAID, MARS.

Particiones inducidas por árboles de decisión

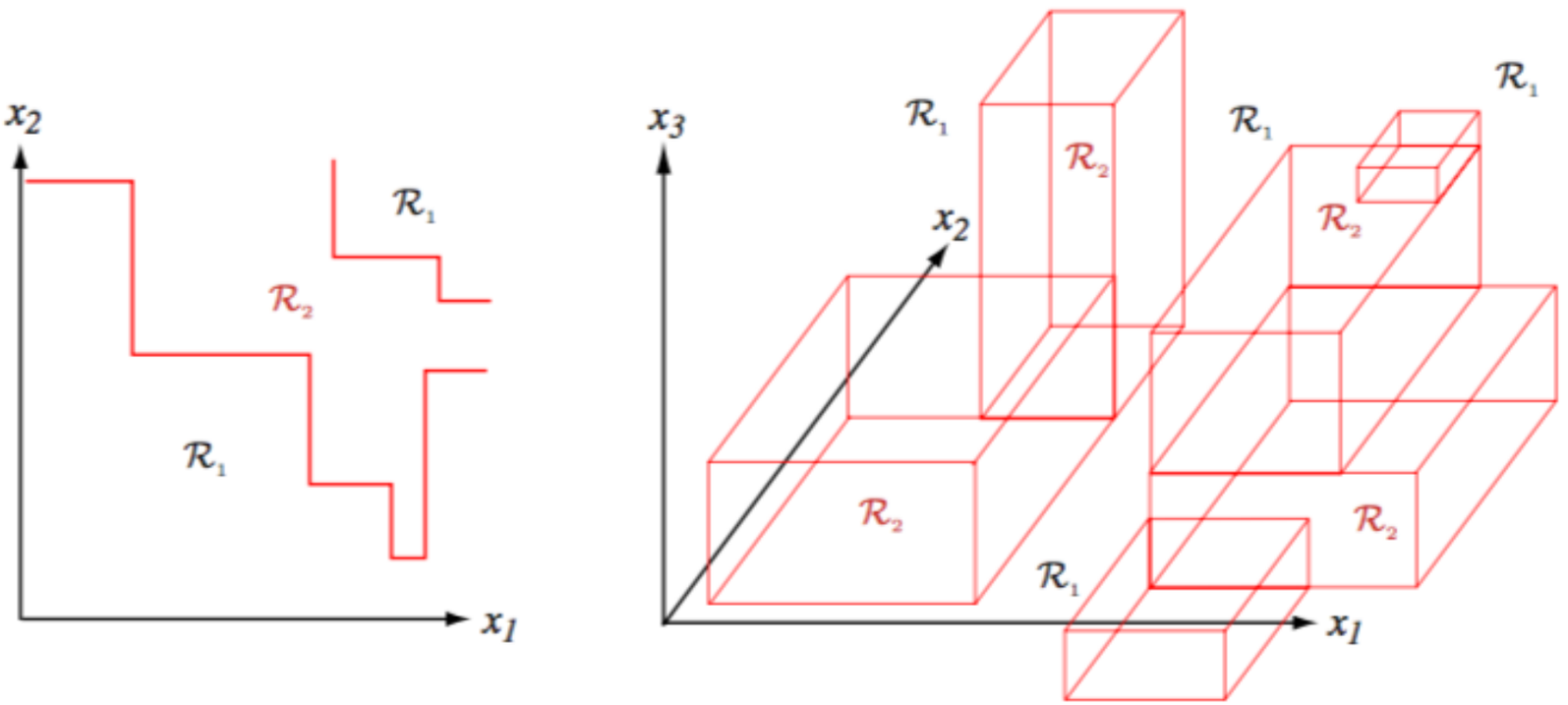
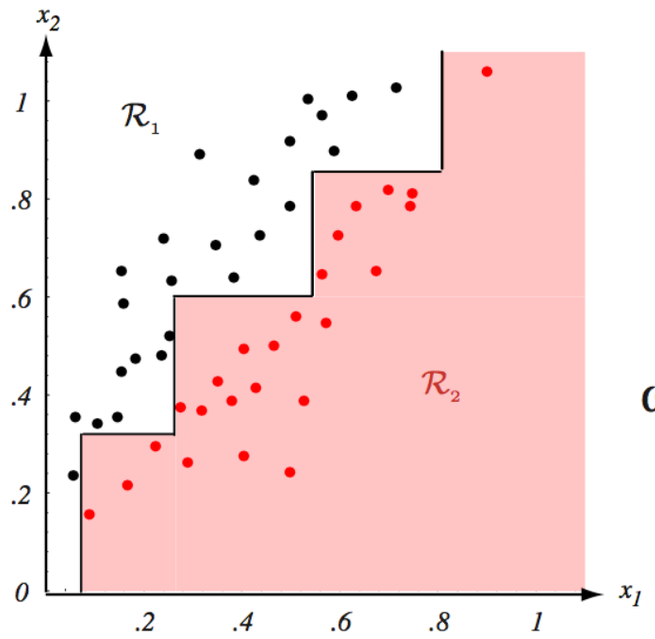
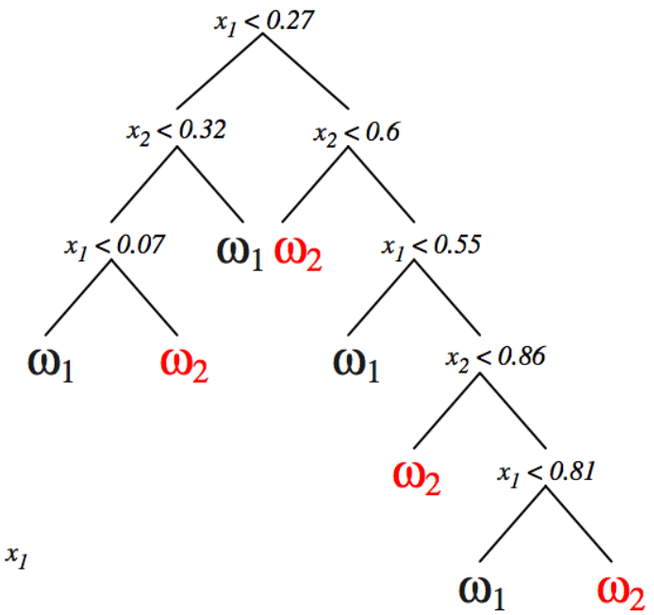


Figure 8.3: Monothetic decision trees create decision boundaries with portions perpendicular to the feature axes. The decision regions are marked $\mathcal{R}_1$ and $\mathcal{R}_2$ in these two-dimensional and three-dimensional two-category examples. With a sufficiently large tree, any decision boundary can be approximated arbitrarily well.

Particiones inducidas por árboles de decisión

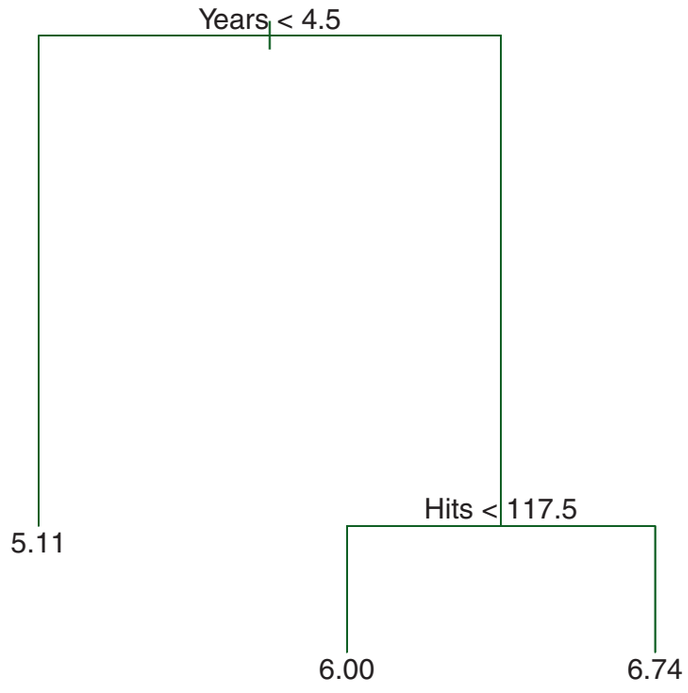




Para un árbol realizar una partición como esta puede ser un verdadero problema

Árboles de Regresión

- La variable objetivo es continua.
- Básicamente los conceptos son los mismos, con la salvedad de que cada nodo hoja se etiqueta con una valor constante.



Árbol para predecir el salario de un jugador en base al número de años jugados y tantos anotados año anterior.

- En este caso cada nodo se etiqueta, tras el proceso de aprendizaje, con la media de los valores asignados a él.

Árboles de Regresión

- Aprendizaje:
  - Comenzaremos por un nodo que “aglutina” todos los ejemplos de entrenamiento.
  - Supondremos división binaria por simplicidad.
  - Tenemos que seleccionar el predictor (variable independiente x_j) y valor de corte (s) para realizar la división de los ejemplos de entrenamiento:

$$R_1(j,s) = \{X | X_j < s\} \text{ and } R_2(j,s) = \{X | X_j \geq s\}$$

- La idea es buscar el óptimo en j y s que minimice el error cuadrático de la predicción:

$$\sum_{i:\, x_i \in R_1(j,s)} (y_i - \hat{y}_{R_1})^2 + \sum_{i:\, x_i \in R_2(j,s)} (y_i - \hat{y}_{R_2})^2$$

Etiquetas asignadas a cada región

Árboles de Regresión

- Búsqueda Exhaustiva de Combinaciones (j, s):
  - Se exploran todas las variables x_j en el conjunto de características X y todos los posibles valores de umbral s dentro de esa variable.
  - Para cada combinación de x_j y s:
    - Se generan los conjunto $R_1(j, s)$m y $R_2(j, s)$.
    - Se calcula el MSE asociado.
  - El par (j, s) que genera el MSE mínimo se selecciona como la división óptima.
- Valores de S:
  - Para cada variable x_j, los posibles valores de s suelen ser los puntos intermedios entre valores consecutivos de x_j en los datos.

$$s \in \left\{ \frac{x_{j,i} + x_{j,i+1}}{2} \;:\; i = 1, \ldots, n_j - 1 \right\}$$